

안녕하세요, 임베디드 개발자 최민서입니다.

Automotive Embedded · AUTOSAR · Arduino

자동차 임베디드 소프트웨어에 관심을 가지고 AUTOSAR 기반 BMS 설계부터 Arduino 기반 자율주행 차량 제어까지 직접 구현하며 실무 역량을 키워왔습니다. 모듈 경계를 먼저 설계하고, 인터페이스를 정의한 뒤 구현하는 접근을 중요하게 생각합니다.

2

부트캠프 수료

3

프로젝트 완주

전자공학

전공

- **스테이지어스** — 웹 개발로 시작, MVC로 구조 다시 잡은 경험
- **모비우스 · BMS** — AUTOSAR 표준 첫 도전, SWC 4개 단독 담당
- **개인 프로젝트** — EdgeCar-FreeRTOS로 같은 원칙을 다른 환경에서 재검증

AUTOSAR Classic

Embedded C / C++

State Machine

CAN 통신

Arduino

Sensor Control

Fault Diagnosis

Moving Avg Filter

BMS FreeRTOS Scheduler

AUTOSAR라는 틀을 벗어나, 같은 BMS 로직을 범용 RTOS로 다시 검증한 개인 프로젝트

6

TASK 파이프라인

55%

스택 실측 절감

0→6

무출력 스케줄러 원인 특정

1인

단독 설계·구현

FreeRTOS Kernel

Cortex-M3 / QEMU

CMake · Ninja

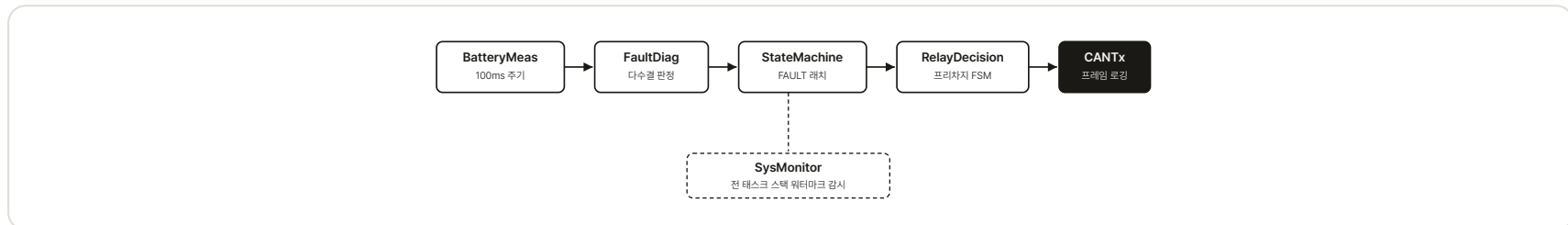
Queue-based IPC

ARM Semihosting

Binary-level Debugging

- 빌드 인프라 구축 — 툴체인·링크스크립트·startup 코드 단독 작성
- 6-태스크 큐 파이프라인 설계 — 데이터 성격별 큐 depth 차등 설계
- FaultDiag 구현 — 다수결 기반 노이즈 필터링 진단
- StateMachine/RelayDecision 구현 — FAULT 래치, 프리차지 FSM
- 링크 레벨 버그 진단 — 심볼 분석으로 스케줄러 정지 원인 특정
- 스택 크기 튜닝 — 실측 워터마크 기반 55% 절감

01 시스템 아키텍처



02 배운 점

- 크래시 로그 없이 조용히 멈춘 스케줄러 — 바이너리 심볼을 직접 대조해 원인을 찾게 됨/!>
- 순간값 대신 다수결 판정으로 노이즈에 흔들리지 않는 상태 판단 설계
- 감으로 잡던 스택 크기를 실측값 기준으로 조정하는 습관을 들임

Battery Management System

AUTOSAR 기반 BMS 아키텍처 설계 및 SWC 구현

4

SWC 단독 담당

5

-State FSM

5

Fault 유형 추적

AUTOSAR Classic

RTE Port Interface

Embedded C

5-State FSM

Bitmask Fault Code

TRK-MPC5606B

CAN Communication

- **SWC 아키텍처 설계** — RTE 포트 인터페이스 기반 4개 모듈 분리
- **BatteryMeasurement 구현** — 상태 기반 전류 유효성 검증
- **FaultDiagnosis 구현** — 비트마스크 기반 다중 Fault 동시 추적
- **BMSStateMachine 구현** — 5-state FSM, Fault 최우선 전이
- **RelayDecision 구현** — 4중 조건 검증, Defense in Depth
- **CAN Tx/Rx 핸들러 구현** — VCU와 신호 송수신 처리

01 시스템 아키텍처



02 배운 점

- 릴레이 제어에 이중 검증을 넣어 단일 실패가 시스템 전체로 번지지 않게 함
- Fault를 우선 처리하는 FSM 설계 — 어느 상태에서든 즉시 전이
- 비트마스크로 다중 Fault를 동시에 표현하는 방식의 이점을 체감
- BMS-VCU 간 CAN 통신으로 신호 송수신 구조를 직접 검증한 경험

EdgeCar

Arduino 기반 4WD 자율주행 차량 제어 시스템

3

계층 아키텍처

6

모듈 단독 구현

1인

기획·구현 전담

Arduino Uno

C / C++

3-Layer Architecture

HC-SR04

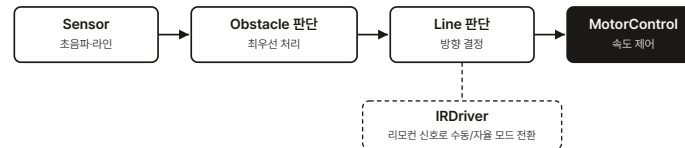
L298N Motor Driver

IR Remote Control

Priority Control

- - **3계층 아키텍처 설계** — Driver / Logic / Control 역할 분리
- - **UltrasonicDriver 구현** — 타임아웃 에러 명시적 처리
- - **LineTrackingDriver 구현** — 3채널 입력 추상화
- - **IRDriver 구현** — 라이브러리 종속성 캡슐화
- - **ObstacleDetection/LineTracking 구현** — 우선순위 기반 회피 로직
- - **MotorControl/ModeManager 구현** — IR 리모컨 기반 모드 전환

01 시스템 아키텍처



02 배운 점

- - 하드웨어-소프트웨어 경계에서 발생하는 문제는 코드만 봐서는 안 보임
- - 센서 이상값을 침묵시키지 않고 명시적으로 전달해야 하는 이유를 직접 겪음
- - 여러 판단이 충돌할 때 우선순위를 먼저 정해두는 게 왜 중요한지 체감